



CEBU INSTITUTE OF TECHNOLOGY
U N I V E R S I T Y

IT342-G1 System Integration and Architecture

System Design Document (SDD)

Project Title: ClinicaFlow

Prepared By: Aloria, Sophia Bianca

Version: 0.6

Date: 5/23/2026

Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
0.1	2/21/2026	Sophia Bianca Aloria	Initial draft	Draft
0.2	2/24/2026	Sophia Bianca Aloria	Added API specifications	Review
0.3	2/28/2026	Sophia Bianca Aloria	Updated database design	Review
0.4	2/28/2026	Sophia Bianca Aloria	Added UI/UX designs	Review
0.5	5/23/2026	Sophia Bianca Aloria	Incorporated feedback	Revised
0.6	5/23/2026	Sophia Bianca Aloria	Final review and corrections	Final
1	5/27/2026		Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY	4
1.0 INTRODUCTION	5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION	5
3.0 NON-FUNCTIONAL REQUIREMENTS	8
4.0 SYSTEM ARCHITECTURE	9
5.0 API CONTRACT & COMMUNICATION	10
Authentication Endpoints	11
User Registration	11
User Login	11
6.0 DATABASE DESIGN	13
7.0 UI/UX DESIGN	14
8.0 PLAN	17

EXECUTIVE SUMMARY

1.1 Project Overview

The Patient Queue & Consultation Management System is a web-based application designed for small clinics centers to efficiently manage patient registration, queueing, and consultation records.

The system helps healthcare providers organize patient flow, reduce manual paperwork, minimize errors, and maintain digital consultation records. It ensures faster service, better patient tracking, and easier access to patient history for future visits.

1.2 Objectives

1. Develop a fully functional web-based clinic management system with secure staff authentication.
2. Design a user-friendly and responsive web interface suitable for clinic staff use.
3. Implement a real-time patient queue management feature with status tracking (Waiting → Consulting → Done).
4. Enable digital recording and storage of consultation notes linked to patient visit history.
5. Allow consultants to write and save consultation notes digitally for better record-keeping.

1.3 Scope

Included Features:

- Staff authentication and role-based access control.(email/password)
- Patient registration and management (add, edit, delete, view, search)
- Medical staff management (add, edit, delete, assign patients)
- Real-time patient queue tracking with status updates (Waiting → Consulting → Done)
- Digital consultation notes recording (Symptoms, Diagnosis, Medicine Prescribed, Remarks)
- Automatic saving of consultation records to patient visit history
- Visit history search and filtering by patient name or consultation date

- Native Android mobile applications for clinic staff and patient access for filing out forms.
- MySQL database (managed using MySQL Workbench) with relational schema for structured and secure data storage

Excluded Features:

- Emergency case prioritization and queue reordering
- SMS or email notification system
- Integration with national health or insurance databases
- Pharmacy or inventory management automation
- Advanced analytics and reporting dashboard
- No Profile for patients and in Mobile App, patients does not need account
- No payment processes
- No Doctor Schedules/Appointments just queueing
- No Digital Prescriptions to Pharmacy
- No Lab Results Integration
- Multi-Language Support

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the Patient Queue & Consultation Management System. It outlines the detailed functional and technical requirements, architectural decisions, database schema, and implementation plan to guide development. The purpose is to ensure that all system components—web interface, mobile application, and backend services—are developed consistently, integrate seamlessly, and provide clinic staff with an efficient, secure, and user-friendly platform for patient registration, queue management, and digital consultation record keeping.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: ClinicaFlow

Domain: Clinic Management

Primary Users:

1. Clinic Staff (Receptionists, Consultants/Doctors)
2. Administrators

Problem Statement: Small clinics need a simple, efficient way to manage patient registration, queue tracking, and consultation record-keeping without relying on manual paperwork.

Solution: A web-based system with a native Android mobile application for staff that provides real-time patient queue management, digital consultation notes, and easy access to patient history.

2.2 Core User Journeys

Journey 1: Patient Registration and Queueing

1. Staff logs in with secure credentials.
2. Staff views the real-time queue dashboard.
3. Staff tracks all patients currently in the queue.
4. Staff updates the status of a patient as they move through the consultation process: **Waiting** → **Consulting** → **Done**.
5. Queue progress is reflected in real-time for all staff.

Journey 2: Consultation Notes Management

1. Consultant begins consultation with a patient marked as “**Consulting**.”
2. Consultant adds new consultation notes, including **Symptoms, Diagnosis, Medicine Prescribed, Remarks**.
3. The consultant can edit or update notes as needed during the session.
4. Notes are securely saved in the patient’s digital record.

Journey 3: Visit History and Record Access

1. Staff searches for a patient by name.
2. Staff retrieves the patient’s consultation notes and visit history.
3. Staff can view, update.

Journey 3: Patient Self-Registration for Queue

1. The patient opens the clinic web page or mobile app.
2. Patient fills out a **simple registration form** with basic information:
 - Name
 - Age
 - Contact Number
 - Reason for visit / Symptoms (optional)
3. The patient submits the form.
4. The system automatically generates a queue number.
5. The patient sees a confirmation page displaying the queue number.

2.3 Feature List (MoSCoW)

MUST HAVE

1. Staff authentication (login, logout)
2. Patient self-registration (simple form for queue)
3. Automatic queue number generation
4. Real-time patient queue tracking (Waiting → Consulting → Done)
5. Digital consultation notes recording (Symptoms, Diagnosis, Medicine, Remarks)
6. Visit history search and filtering (by patient name or consultation date)
7. Secure storage of consultation notes

SHOULD HAVE

1. Responsive web and mobile interface for staff and patients
2. Edit/update patient details and consultation notes

COULD HAVE

1. Estimated waiting time for patients in the queue
2. Dashboard summary of daily patient visits
3. Quick search/filter for queue dashboard

WON'T HAVE

1. Patient login or accounts
2. SMS/email notifications for queue updates
3. Integration with national health/insurance databases
4. Pharmacy, lab results, or inventory management
5. Appointment booking system
6. Emergency queue.

2.4 Detailed Feature Specifications

Feature: Staff Authentication

- **Screens:** Login, Logout
- **Fields:** Email, Password
- **Validation:** Email format, password strength
- **API Endpoints:** POST /auth/login, POST /auth/logout
- **Security:** JWT tokens, password hashing

Feature: Patient Self-Registration

- User fills out registration form
- Submit form → System calls /api/patient/register-queue
- System automatically generates:
 - Queue Number
 - Queue Position
 - Estimated Wait Time
 - Assigned Doctor
- User receives confirmation with queue details

Method	Endpoint	Description
POST	/api/patient/register-queue	Register patient to queue
GET	/api/patient/queue	Get active queue (all patients)
GET	/api/patient/queue/all	Get full queue (all patients including completed)
GET	/api/patient/all	Get all patients (for Patient Records page)
GET	/api/patient/queue-status	Get queue status (length, wait time, current queue)
PUT	/api/patient/{id}	Update patient by ID
PATCH	/api/patient/{id}/status	Update patient status only
DELETE	/api/patient/{id}	Delete patient (permanent)
DELETE	/api/patient/{id}/permanent	Permanent delete (admin)
GET	/api/patient/test	Test endpoint

- **Notes:** No patient account or login required

Feature: Consultation Notes Management

- **Screens:** Consultation Form
- **Fields:** Symptoms, Diagnosis, Medicine Prescribed, Remarks
- **Functions:** Add new notes, edit/update notes, save securely to patient record
- **API Endpoints:**

Method	Endpoint	Description
POST	/api/consultations/add	Create new consultation record
GET	/api/consultations/all	Get all consultation records
GET	/api/consultations/{id}	Get consultation by ID
GET	/api/consultations/date/{date}	Get consultations by date (format: ISO date)
GET	/api/consultations/doctor/{doctorName}	Get consultations by doctor name
GET	/api/consultations/today/count	Get today's consultation count
PUT	/api/consultations/update/{id}	Update consultation by ID
DELETE	/api/consultations/delete/{id}	Delete consultation by ID

Feature: Medical Staff Management & Profile

- **Screens**
 - Staff Management Dashboard
 - Add/Edit Staff Form
 - Staff Profile View
 - Staff Photo Upload

- **API Endpoints:**
Staff CRUD Operations

Method	Endpoint	Description
POST	/api/medicalstaff/add	Add new staff member
GET	/api/medicalstaff/all	Get all staff members
GET	/api/medicalstaff/{id}	Get staff by ID
PUT	/api/medicalstaff/update/{id}	Update staff information
DELETE	/api/medicalstaff/delete/{id}	Delete staff member

User Account Integration

Method	Endpoint	Description
POST	/api/medicalstaff/sync-from-users	Sync DOCTOR users to staff table
GET	/api/medicalstaff/users	Get all user accounts
GET	/api/medicalstaff/user/by-email/{email}	Get user by email
GET	/api/medicalstaff/by-account/{accountId}	Get staff by account ID

Search & Filter & Photo Management

Method	Endpoint	Description
GET	/api/medicalstaff/role/{role}	Get staff by role
GET	/api/medicalstaff/specialty/{specialty}	Get staff by specialty
POST	/api/medicalstaff/upload-photo	Upload staff photo

Feature: Admin Panel

- **Screens:**
 - Staff Management (Add/Edit/Delete Staff)
 - System Dashboard (queue and consultation overview)
 - Visit History Overview (search and filter all patients)
- **Functions:**
 - Manage clinic staff accounts (create, edit, delete)
 - Monitor real-time patient queue status
 - Access and review consultation notes for auditing

- **API Endpoints:**

GET	/admin/dashboard	Access patient records and staffs
GET	/admin/history	access patient visit history

- **Access Control:** Only admin users can access; regular staff cannot

2.5 Acceptance Criteria

AC-1: Successful Staff Login

- Given a registered staff member
- When valid credentials are entered
- Then the staff member should be logged in and redirected to the dashboard

AC-2: Patient Queue Registration

- Given a patient accesses the registration form
- When the patient submits valid details
- Then a queue number is automatically generated
- And the patient sees their queue number

AC-3: Queue Status Update

- Given patients are in the queue
- When staff updates a patient's status to **Consulting or Done**

- Then the queue dashboard reflects the status in real-time

AC-4: Consultation Notes Recording

- Given a patient is being consulted
- When the consultant enters symptoms, diagnosis, medicine, and remarks
- And saves the notes
- Then the notes are stored securely and appear in the patient's visit history

AC-4: Visit History Search

- Given staff searches for a patient by name or date
- When a valid patient record is found
- Then the consultation notes are displayed

3.0 NON-FUNCTIONAL REQUIREMENTS

2.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests
- Web page load time: ≤ 3 seconds on broadband
- Mobile app cold start: ≤ 3 seconds
- Support 100 concurrent users
- Database queries complete within 500ms

2.2 Security Requirements

- HTTPS for all communications
- JWT token authentication
- Password hashing with bcrypt (salt rounds = 12)
- SQL injection prevention
- XSS protection
- Rate limiting: 100 requests/minute per IP
- Admin endpoints require role verification

2.3 Compatibility Requirements

- **Web Browsers:** Chrome, Firefox, Safari
- **Android:** API Level 24+ (Android 7.0+)
- **Screen Sizes:** Mobile (360px+), Tablet (768px+), Desktop (1024px+)
- **Operating Systems:** Windows 10+

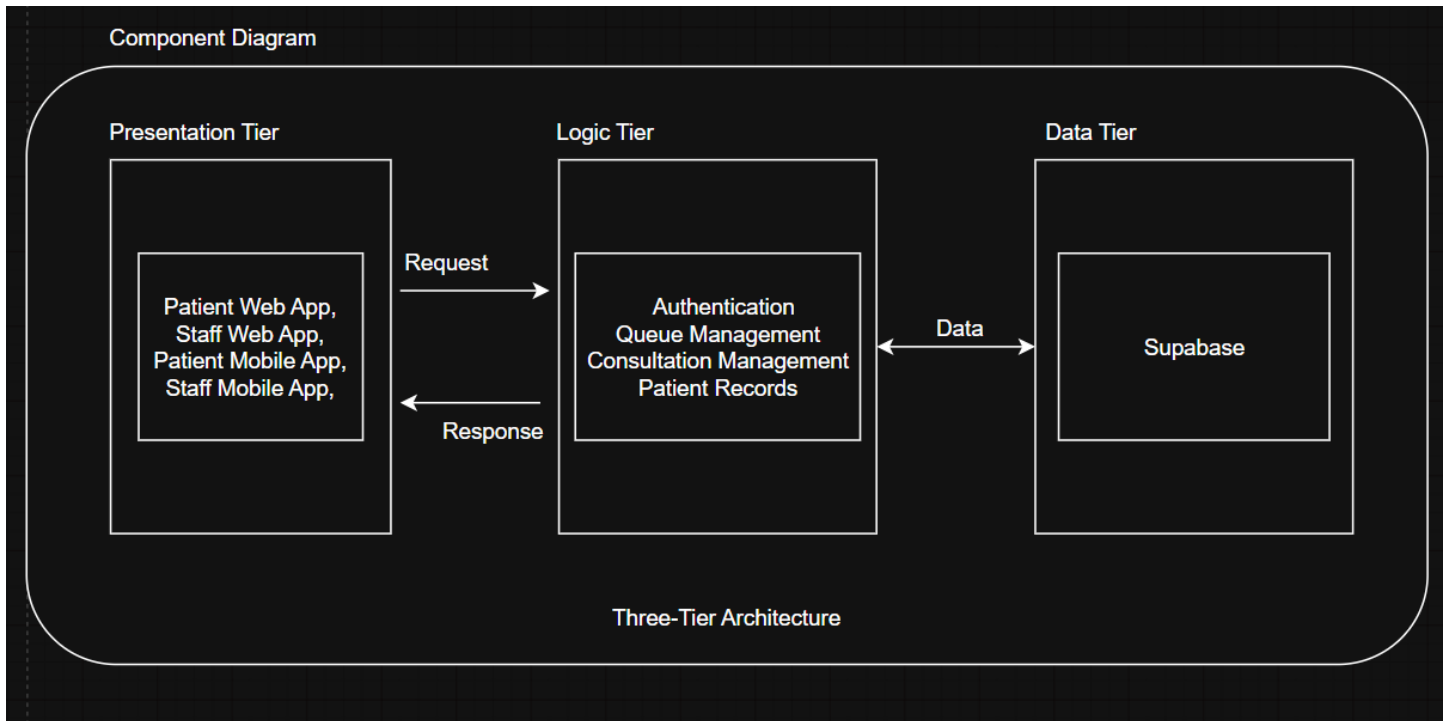
2.4 Usability Requirements

- WCAG 2.1 Level AA compliance for web
- Consistent navigation across all pages
- Clear error messages with recovery options

- Touch targets minimum 44x44px on mobile

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
- **Database:** Supabase
- **Web Frontend:** React, TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android)
- **Deployment:** Vercel/Netlify (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	http://localhost:3000
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header

Response Structure	<pre>{ "success": boolean, "data": object null, "error": { "code": string, "message": string, "details": object null }, "timestamp": string }</pre>
--------------------	---

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/api/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Supabase Email verification
Request Payload	{ "firstName": "Juan", "lastName": "Dela Cruz", "email": "juan@email.com", "password": "password123", "role": "STAFF" }
Response Structure	{ "success": true, "message": "User registered successfully", "token": "jwt_token_here", "user": { "id": 1, "email": "juan@email.com", "role": "STAFF" } }

User Login

Description	User Login
API URL	/api/auth/login
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Verified Email or Google Authentication
Request Payload	{ "email": "user@email.com", "password": "password123" }
Response Structure	{ "success": true, "data": { "user": { "email": "user@email.com", "firstname": "Juan", "lastname": "Dela Cruz", "role": "STAFF" }, "accessToken": "jwt_access_token_here", "refreshToken": "jwt_refresh_token_here" }, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Resend Verification Email

Description	Resend email verification link
API URL	/api/auth/resend-verification
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	{ "email": "user@email.com" }
Response Structure	{ "success": true, "message": "Verification email resent successfully", "data": null, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Confirm Email

Description	Confirm user email address
API URL	/api/auth/confirm-email
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	{ "email": "user@email.com", "token": "confirmation_token" }
Response Structure	{ "success": true, "message": "Email verified successfully", "data": { "emailVerified": true, "email": "user@email.com" }, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Change Password

Description	Change user password
API URL	/api/auth/change-password
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	{ "email": "user@email.com", "currentPassword": "old123", "newPassword": "new123" }
Response Structure	{ "success": true, "message": "Password changed successfully", "data": null, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Logout All Devices

Description	Logout from all devices
API URL	/api/auth/logout-all
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	None
Response Structure	{ "success": true, "message": "Logged out from all devices", "data": null, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Patient Endpoints

Patient Self-Registration (Queue)

Description	Patient registers to queue without account
API URL	/api/patient/register-queue
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None
Request Payload	{ "firstName": "Juan", "lastName": "Dela Cruz", "age": 30, "gender": "Male", "contactNumber": "09123456789", "address": "123 Street" }
Response Structure	{ "success": true, "data": { "queueNumber": "Q001", "queuePosition": 1, "estimatedWaitTime": "15 minutes", "assignedDoctor": "Dr. Smith" }, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Get Queue Status

Description	Get current queue status and statistics
API URL	/api/patient/queue-status
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	None
Request Payload	None
Response Structure	{ "success": true, "data": { "queueLength": 5, "estimatedWaitTime": "45 minutes", "currentQueue": [] }, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Get All Patients

Description	Get all patient records
API URL	/api/patient/all
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	None
Response Structure	{ "success": true, "data": [{ "id": "PAT001", "firstName": "Juan", "lastName": "Dela Cruz", "status": "completed" }], "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Get Patient by ID

Description	Get specific patient by ID
API URL	/api/patient/{id}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	None

Response Structure	{ "success": true, "data": { "id": "PAT001", "firstName": "Juan", "lastName": "Dela Cruz", "age": 30, "status": "waiting" }, "error": null, "timestamp": "2026-02-27T14:30:00Z" }
--------------------	---

Update Patient

Description	Update patient information
API URL	/api/patient/{id}
HTTP Request Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	{ "firstName": "Juan", "lastName": "Dela Cruz", "age": 31, "contactNumber": "09987654321" }
Response Structure	{ "success": true, "message": "Patient updated successfully", "data": { ... }, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Update Patient Status

Description	Update patient status only
API URL	/api/patient/{id}/status
HTTP Request Method	PATCH
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	{ "status": "consulting", "assignedDoctor": "Dr. Smith" }
Response Structure	{ "success": true, "message": "Patient status updated successfully", "data": { ... }, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Delete Patient

Description	Permanently delete patient record
API URL	/api/patient/{id}
HTTP Request Method	DELETE
Format	JSON for all requests/responses
Authentication	Bearer token required (Admin only)
Request Payload	None
Response Structure	{ "success": true, "message": "Patient permanently deleted from records", "data": null, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Consultation Endpoints

Add Consultation

Description	Add new consultation notes
API URL	/api/consultations/add
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	{ "patientId": "PAT001", "patientName": "Juan Dela Cruz", "age": 30, "gender": "Male", "doctorName": "Dr. Smith", "consultationDate": "2026-05-23", "symptoms": "Fever and cough", "diagnosis": "Acute URI", "medicinePrescribed": "Paracetamol 500mg", "remarks": "Rest for 3 days" }
Response Structure	{ "success": true, "data": { "id": 1, "patientId": "PAT001", ... }, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Get All Consultations

Description	Get all consultation records
API URL	/api/consultations/all
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	None
Response Structure	{ "success": true, "data": [{ "id": 1, "patientId": "PAT001", "patientName": "Juan Dela Cruz", "doctorName": "Dr. Smith", "consultationDate": "2026-05-23", "symptoms": "Fever", "diagnosis": "URI", "prescription": "Paracetamol", "remarks": "Rest" }], "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Get Consultation by ID

Description	Get consultation by ID
API URL	/api/consultations/{id}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	None
Response Structure	{ "success": true, "data": { "id": 1, "patientId": "PAT001", ... }, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Get Consultations by Date

Description	Get consultations by specific date
API URL	/api/consultations/date/{date}
HTTP Request Method	GET

Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	None
Response Structure	{ "success": true, "data": [{ "id": 1, "consultationDate": "2026-05-23", ... }], "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Get Consultations by Doctor

Description	Get consultations by doctor name
API URL	/api/consultations/doctor/{doctorName}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	None
Response Structure	{ "success": true, "data": [{ "id": 1, "doctorName": "Dr. Smith", ... }], "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Get Today's Consultation Count

Description	Get number of consultations for current date
API URL	/api/consultations/today/count
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	None
Response Structure	{ "success": true, "data": 15, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Update Consultation

Description	Update existing consultation record
API URL	/api/consultations/update/{id}
HTTP Request Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	{ "symptoms": "Updated symptoms", "diagnosis": "Updated diagnosis", "medicinePrescribed": "New medicine",

	"remarks": "Updated remarks" }
Response Structure	{ "success": true, "data": { "id": 1, ... }, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Delete Consultation

Description	Delete consultation record
API URL	/api/consultations/delete/{id}
HTTP Request Method	DELETE
Format	JSON for all requests/responses
Authentication	Bearer token required (Admin only)
Request Payload	None
Response Structure	{ "success": true, "message": "Consultation deleted successfully", "data": null, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Medical Staff Endpoints

Sync Staff from Users

Description	Sync DOCTOR role users to medical staff table
API URL	/api/medicalstaff/sync-from-users
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token required (Admin only)
Request Payload	None
Response Structure	{ "success": true, "data": { "created": 3, "updated": 2 }, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Add Staff Member

Description	Add new medical staff member
API URL	/api/medicalstaff/add
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	Bearer token required (Admin only)
Request Payload	{ "fname": "Sarah", "lname": "Johnson", "email": "sarah@clinicaflow.com", "role": "Doctor", "specialty": "Cardiology", "contactNo": "09123456789", "availability": "Available" }
Response Structure	{ "success": true, "data": { "staffID": 5, ... }, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Get All Staff

Description	Get all medical staff members
API URL	/api/medicalstaff/all
HTTP Request	GET

Method	
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	None
Response Structure	{ "success": true, "data": [{ "staffID": 1, "fname": "John", "lname": "Smith", "role": "Doctor" }], "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Get Staff by ID

Description	Get staff member by ID
API URL	/api/medicalstaff/{id}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	None
Response Structure	{ "success": true, "data": { "staffID": 1, "fname": "John", "lname": "Smith" }, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Get Staff by Role

Description	Get staff members by role
API URL	/api/medicalstaff/role/{role}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	None
Response Structure	{ "success": true, "data": [{ "staffID": 1, "role": "Doctor" }], "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Get Staff by Specialty

Description	Get staff members by specialty
API URL	/api/medicalstaff/specialty/{specialty}
HTTP Request Method	GET
Format	JSON for all requests/responses
Authentication	Bearer token required
Request Payload	None
Response Structure	{ "success": true, "data": [{ "staffID": 1, "specialty": "Cardiology" }], "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Update Staff

Description	Update staff member information
-------------	---------------------------------

API URL	/api/medicalstaff/update/{id}
HTTP Request Method	PUT
Format	JSON for all requests/responses
Authentication	Bearer token required (Admin only)
Request Payload	{ "fname": "Sarah", "specialty": "Pediatrics", "availability": "On Leave" }
Response Structure	{ "success": true, "data": { "staffID": 5, ... }, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Delete Staff

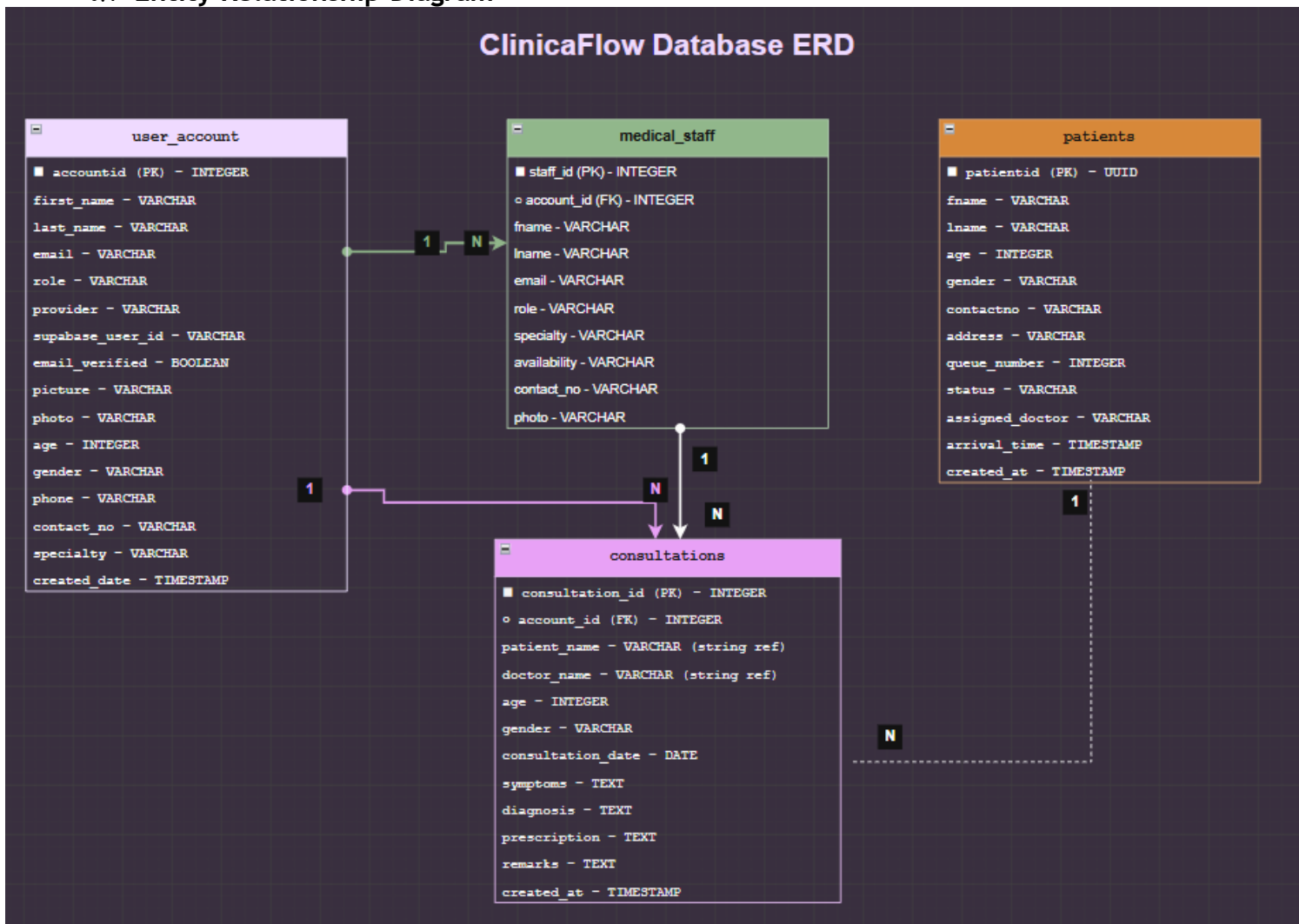
Description	Delete staff member
API URL	/api/medicalstaff/delete/{id}
HTTP Request Method	DELETE
Format	JSON for all requests/responses
Authentication	Bearer token required (Admin only)
Request Payload	None
Response Structure	{ "success": true, "message": "Medical staff with ID 5 has been deleted.", "data": null, "error": null, "timestamp": "2026-02-27T14:30:00Z" }

Upload Staff Photo

Description	Upload profile photo for staff member
API URL	/api/medicalstaff/upload-photo
HTTP Request Method	POST
Format	multipart/form-data
Authentication	Bearer token required
Request Payload	file: (image file, max 5MB), email: "sarah@clinicaflow.com"
Response Structure	{ "success": true, "photoUrl": "http://localhost:8080/uploads/staff_photos/123.jpg", "message": "Photo uploaded successfully", "error": null, "timestamp": "2026-02-27T14:30:00Z" }

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- One-to-One: **user_account** ↔ **medical_staff** (Each staff member has exactly one user account)
- One-to-Many: **user_account** → **consultations** (One user account can create multiple consultations)
- One-to-Many: **patients** → **consultations** (One patient can have multiple consultations)
- One-to-Many: **medical_staff** → **consultations** (One medical staff can handle multiple consultations)

Key Tables:

1. **UserAccount** - User accounts and authentication
2. **Patient** - Patient personal information
3. **MedicalStaff** - Staff personal and professional information
4. **VisitHistory** - Records of patient visits
5. **Consultation** - Consultation details per visit
6. **Queue** - Patient queue management

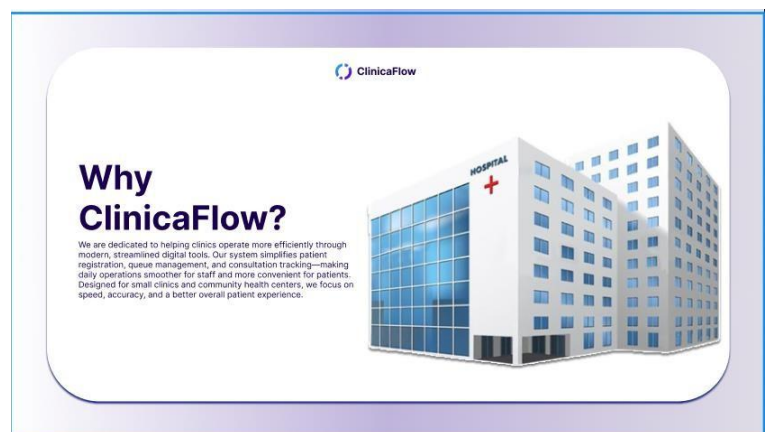
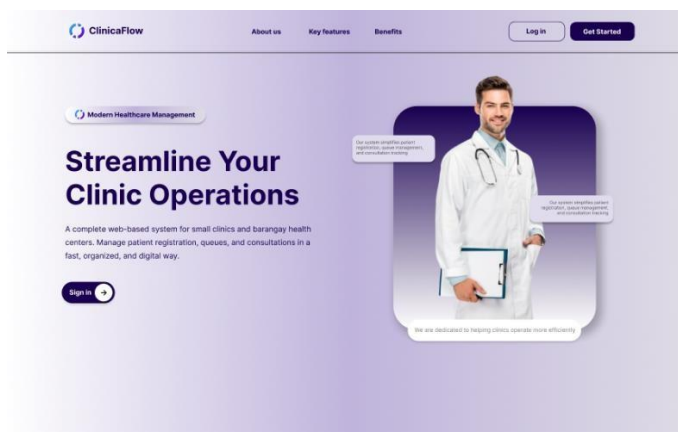
Table Structure Summary:

- **user_account:** accountid, first_name, last_name, email, password_hash, role, provider, supabase_user_id, email_verified, picture, photo, age, gender, phone, contact_no, specialty, availability, department, created_date, last_login
- **patients:** patientid, fname, lname, age, gender, contactno, address, queue_number, status, assigned_doctor, arrival_time, created_at, updated_at
- **medical_staff:** staff_id, account_id, fname, lname, email, role, specialty, availability, contact_no, photo, created_at, updated_at
- **consultations:** consultation_id, account_id, patient_name, doctor_name, age, gender, consultation_date, symptoms, diagnosis, prescription, remarks, created_at

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Landing Page



Highlights & Key Features

Patient Management

Easily add, update, and view patient records in one centralized system. Monitor complete and accurate information about each patient — including contact details, medical history, and real-time logs. This feature eliminates the need for manual paper records and streamlines every consultation it's backed by accessible patient data.





Queue Tracking

Monitor and manage in-the-patient queue efficiently in real time. The system automatically updates queue status, allowing staff to see when each patient has been waiting. This ensures a smoother clinic flow, reduces patient frustration, and optimizes staff scheduling.

Consultation Records

Keep a digital history of patient consultations, securely stored and easy to retrieve. Doctors can record diagnoses, treatments, and prescriptions with just a few clicks. This makes it simple to review past visits, support better decision-making, and provide personalized patient care.



What You Gain

Our platform reduces waiting times, improves data accuracy, and helps clinics deliver faster, more organized patient care. With real-time queue updates, secure digital records, and a simplified workflow, staff can work more efficiently while patients enjoy a smoother and more convenient clinic experience.

Skip the Line. Join Us Online

Register yourself in the patient queue from the comfort of your home. No need to wait at the clinic - we'll notify you when it's your turn.

Join Queue Now

You'll receive a queue number immediately after registration

Login and Sign in Page

sign in



Digital Health Management Made Simple

Easily manage patient records, queues, and consultations — all in one place.

Start Managing Patients the Fast, Simple, and Digital Way

First Name Last Name

Email

Role

Select your role (Health Dispenser/ty)

Payment

Confirm Payment

☐ I agree to the Terms & Conditions

Don't have an account? [Join right now](#)

login



Digital Health Management Made Simple

Easily manage patient records, queues, and consultations — all in one place.

Welcome back to Your ClinicaFlow Dashboard

Email

Remember me ☐

[No Account yet? SIGN UP](#)

Main Page

ClinicaFlow

Medical Management

Patient Queue

Patients

Medical Staff

Appointments

Medical History

3

Total Patients

2

Waiting

1

Consulting

1

Completed

3 patients in queue

1

Dr. Smith

3

Total Patients

2

Waiting

1

Consulting

1

Completed

3 patients in queue

1

Dr. Smith

ClinicaFlow

Medical Management

Patient Queue

Patients

Medical Staff

Appointments

Medical History

5

Total Patients

3

Waiting

1

Consulting

1

Completed

3 patients in queue

1

Dr. Smith

5

Total Patients

3

Waiting

1

Consulting

1

Completed

3 patients in queue

1

Dr. Smith

ClinicaFlow

Medical Management

Patient Queue

Patients

Medical Staff

Appointments

Medical History

3

Total Patients

2

Waiting

1

Consulting

1

Completed

3 patients in queue

1

Dr. Smith

3

Total Patients

2

Waiting

1

Consulting

1

Completed

3 patients in queue

1

Dr. Smith

ClinicaFlow

Medical Management

Patient Queue

Patients

Medical Staff

Appointments

Medical History

5

Total Patients

3

Waiting

1

Consulting

1

Completed

3 patients in queue

1

Dr. Smith

5

Total Patients

3

Waiting

1

Consulting

1

Completed

3 patients in queue

1

Dr. Smith

ClinicaFlow

Medical Management

Patient Queue

Patients

Medical Staff

Appointments

Medical History

5

Total Patients

3

Waiting

1

Consulting

1

Completed

3 patients in queue

1

Dr. Smith

5

Total Patients

3

Waiting

1

Consulting

1

Completed

3 patients in queue

1

Dr. Smith

25

Profile Page

Account Settings

Manage your account and preferences securely

Home

Profile

Security

Personal Profile

Manage your account and preferences securely

Personal Information

Edit Profile

MC

Dr. Maria Cruz

Senior Physician

Avatar

Age

45 Years

Gender

Female

Email

maria.cruz@gmail.com

Phone

09750708553

Professional Information

Availability Status

Specialization

Dermatology

Department

Physician

Busy

Available

Offline

Account Settings

Manage your account and preferences securely

Home

Profile

Security

Personal Profile

Manage your account and preferences securely

Password Security

Secure your account and data

Change Password

To change your password, please fill in the details below

Current Password

New Password

Confirm Password

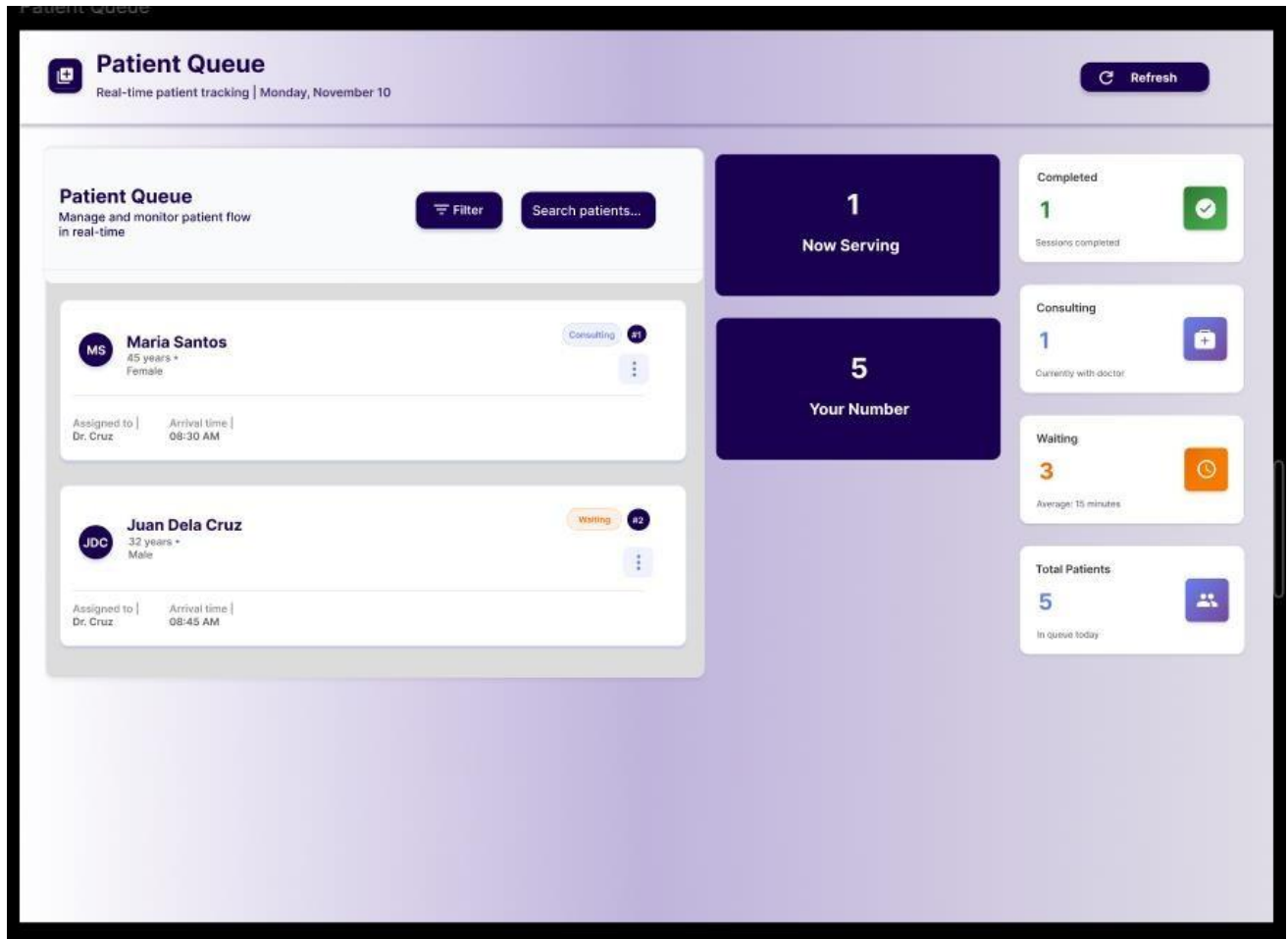
Change Password

Your Devices

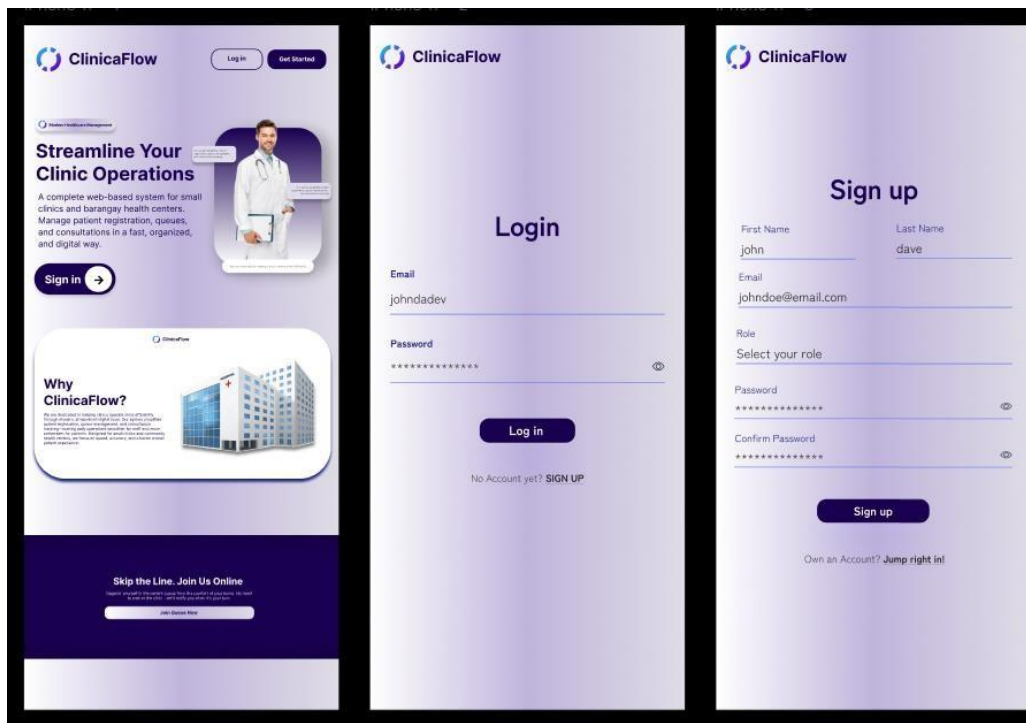
You are logged in on 1 device...

Log Out From All Devices

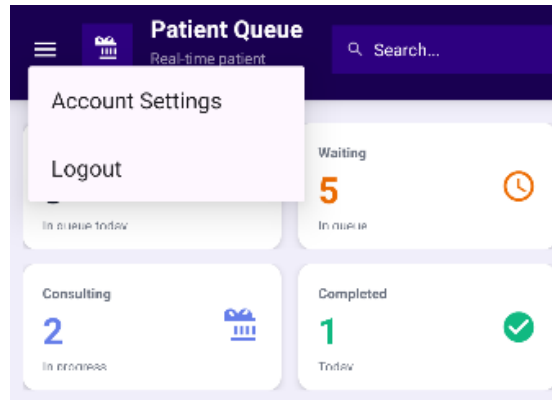
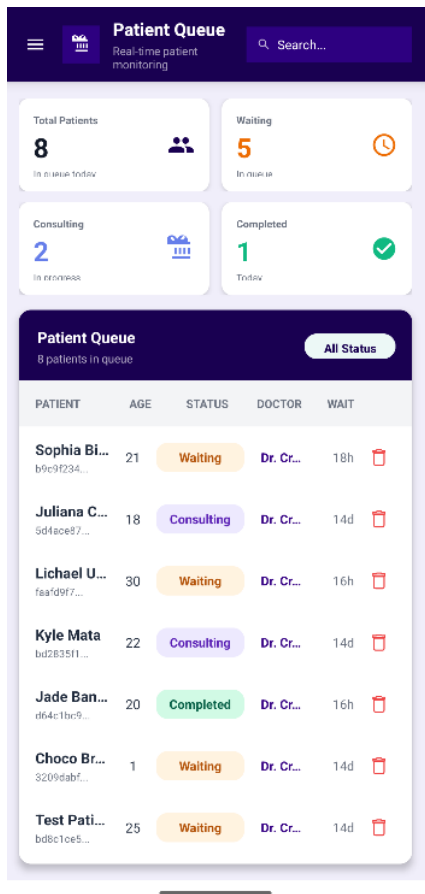
Patient Queue, Join Queue



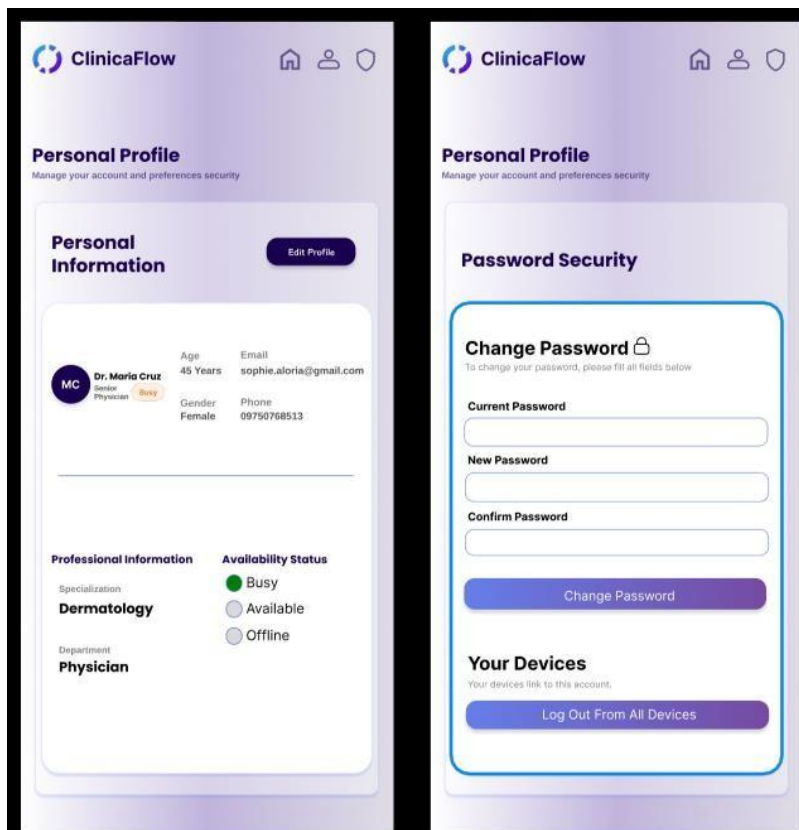
Landing Page, Login and SignUp Page



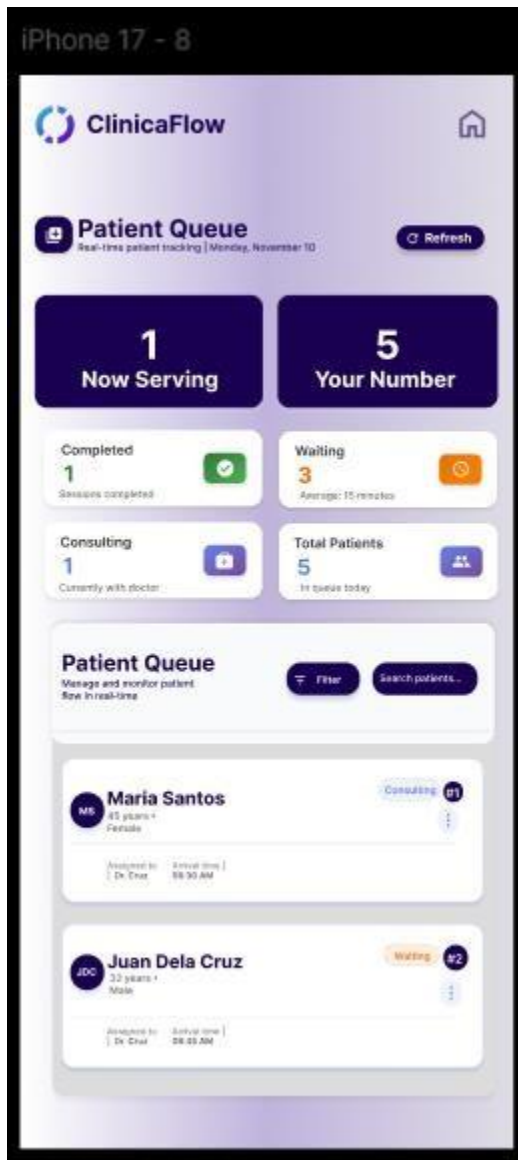
Main Page



Profile Page



Patient Queue, Join Queue



Patient Registration Screen

Back button

Patient information form (First Name, Last Name, Age, Gender, Contact No, Address)

Editable fields for updating patient details

"Save Patient" button

"Assign to Queue" button

Queue Management Screen

Queue number display
Patient list (Waiting / Consulting / Done status)
Status update dropdown (Waiting → Consulting → Done)
Real-time queue list
“Add to Queue” button
Live queue counter

Consultation Screen

Patient basic info header
Symptoms input field
Diagnosis input field
Medicine prescribed field
Remarks section
Consultation date auto-filled
“Save Consultation Notes” button

Visit History Screen

Search bar (Search by patient name or date)
Filter by date
List of past consultations
View full consultation details

Patient Queue Monitoring Screen

Queue number display
Current serving patient indicator
Status tracker (Waiting → Consulting → Done)
Real-time updates panel

Language Settings Screen

Language selection dropdown
Save preference button
Preview interface in selected language

Mobile-Specific Features:

- Touch-optimized buttons (min 44x44px)
- Burger Icon for features
- Simplified forms for mobile input

Design System:

- **Colors:** Primary (#DFDCE6), Secondary (#190051),
- **Typography:** Inter font family, responsive sizing
- **Spacing:** 8px grid system
- **Components:** Consistent buttons, inputs, cards, modals
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024px

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and documentation

Day 3-4: Complete FRS and NFR

Day 5-7: System architecture design

Week 2: Detailed Design

Day 1-2: API specification

Day 3-4: Database design

Day 5-6: UI/UX wireframes

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with dependencies

Day 2: Database configuration and entities

Day 3: JWT authentication implementation

Day 4: User management endpoints

Day 5: Product CRUD operations

Week 4: Core Features

Day 1: Cart functionality

Day 2: Order management

Day 3: Search and filtering

Day 4: Error handling and validation

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup with TypeScript

Day 2: Authentication pages (login, register)

Day 3: Product listing page

Day 4: Product detail page

Day 5: Shopping cart implementation

Week 6: Complete Web Features

Day 1: Checkout flow

Day 2: Order history and confirmation

Day 3: Admin dashboard

Day 4: Responsive design polish

Day 5: API integration and testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup and project structure

Day 2: Authentication screens

Day 3: Product browsing

Day 4: Shopping cart

Day 5: API service layer

Week 8: Complete Mobile App

Day 1: Checkout flow

Day 2: Order management

Day 3: UI polish and animations

Day 4: Testing on emulator/device

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: End-to-end testing across platforms

Day 2: Bug fixes and optimization

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation updates

Week 10: Deployment

Day 1: Backend deployment (Railway/Heroku)

Day 2: Web app deployment (Vercel/Netlify)

Day 3: Mobile APK distribution

Day 4: Final testing

Day 5: Project submission

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Product catalog API (Week 3-4)
3. Shopping cart functionality (Week 4)
4. Checkout process (Week 6)
5. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements